```
import os
import sys
import time
import subprocess
import traceback
from flask import Flask, render_template, request, jsonify, send_from_directory
from werkzeug.utils import secure_filename

from pipeline.dataset import MOVIES_DATA, get_movies, get_movie_by_id, ensure_sample_posters
from pipeline.preprocessor import ImagePreprocessor
from pipeline.feature_extractor import MultimodalFeatureExtractor
from pipeline.model_trainer import ModelTrainer
from pipeline.evaluator import PerformanceEvaluator
from pipeline.recommender import MovieRecommenderEngine

app = Flask(__name__, static_folder="static", template_folder="templates")
app.config["MAX_CONTENT_LENGTH"] = 16 * 1024 * 1024  # 16 MB max upload

# Base directories
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
POSTERS_DIR = os.path.join(BASE_DIR, "static", "sample_posters")
UPLOAD_DIR = os.path.join(BASE_DIR, "static", "uploads")
os.makedirs(UPLOAD_DIR, exist_ok=True)

# Ensure sample posters exist
ensure_sample_posters(POSTERS_DIR)

# Initialize ML Pipeline Components
preprocessor = ImagePreprocessor(target_size=(128, 128))
trainer = ModelTrainer(visual_weight=0.5)
evaluator = PerformanceEvaluator()

# Initial training
print("[STARTUP] Initializing CineMatch ML Pipeline and caching feature matrices...")
trainer.initialize_feature_matrices(POSTERS_DIR)
trainer.train(model_type="random_forest", k_neighbors=5, n_estimators=100)
recommender = MovieRecommenderEngine(trainer)
print("[STARTUP] CineMatch ML Pipeline is ready!")

# Available files for in-browser Code Studio
ALLOWED_CODE_FILES = {
    "app.py": os.path.join(BASE_DIR, "app.py"),
    "pipeline/preprocessor.py": os.path.join(BASE_DIR, "pipeline", "preprocessor.py"),
    "pipeline/feature_extractor.py": os.path.join(BASE_DIR, "pipeline", "feature_extractor.py"),
    "pipeline/model_trainer.py": os.path.join(BASE_DIR, "pipeline", "model_trainer.py"),
    "pipeline/evaluator.py": os.path.join(BASE_DIR, "pipeline", "evaluator.py"),
    "pipeline/recommender.py": os.path.join(BASE_DIR, "pipeline", "recommender.py"),
    "pipeline/dataset.py": os.path.join(BASE_DIR, "pipeline", "dataset.py"),
}

@app.route("/")
def index():
    """Serves the unified one-frame web application."""
    return render_template("index.html")

@app.route("/api/movies", methods=["GET"])
def api_movies():
```

```
    """Returns catalog of movies with metadata."""
    movies = get_movies()
    return jsonify({
        "success": True,
        "count": len(movies),
        "movies": movies
    })

@app.route("/api/preprocess", methods=["POST"])
def api_preprocess():
    """
    Executes 6-stage image preprocessing:
    Accepts:
    - Multipart image file ('image') OR
    - JSON payload with 'image_base64' or 'movie_id'
    """
    try:
        image_input = None

        if "image" in request.files and request.files["image"].filename:
            file = request.files["image"]
            filename = secure_filename(file.filename)
            filepath = os.path.join(UPLOAD_DIR, filename)
            file.save(filepath)
            image_input = filepath
        else:
            data = request.get_json(silent=True) or {}
            if "image_base64" in data and data["image_base64"]:
                image_input = data["image_base64"]
            elif "movie_id" in data and data["movie_id"]:
                movie = get_movie_by_id(data["movie_id"])
                if movie:
                    image_input = os.path.join(POSTERS_DIR, movie["poster_file"])

        if not image_input:
            # Default to first sample poster if nothing provided
            image_input = os.path.join(POSTERS_DIR, "interstellar.png")

        results = preprocessor.process(image_input)

        # Don't serialize non-JSON PIL images and raw arrays
        stages = results["stages"]
        metrics = results["metrics"]

        return jsonify({
            "success": True,
            "stages": stages,
            "metrics": metrics
        })
    except Exception as e:
        traceback.print_exc()
        return jsonify({"success": False, "error": str(e)}), 500

@app.route("/api/extract-features", methods=["POST"])
def api_extract_features():
    """
    Extracts deep visual features from poster image:
```

```
    Returns color histogram, edge map, dominant colors, and feature metrics.
    """
    try:
        data = request.get_json(silent=True) or {}
        image_input = None

        if "image_base64" in data and data["image_base64"]:
            image_input = data["image_base64"]
        elif "movie_id" in data and data["movie_id"]:
            movie = get_movie_by_id(data["movie_id"])
            if movie:
                image_input = os.path.join(POSTERS_DIR, movie["poster_file"])

        if not image_input:
            image_input = os.path.join(POSTERS_DIR, "interstellar.png")

        fe = trainer.feature_extractor
        visual_vec, details = fe.extract_visual_features(image_input)

        return jsonify({
            "success": True,
            "features": details
        })
    except Exception as e:
        traceback.print_exc()
        return jsonify({"success": False, "error": str(e)}), 500

@app.route("/api/train", methods=["POST"])
def api_train():
    """
    Triggers model training with user-specified hyperparameters.
    """
    try:
        data = request.get_json(silent=True) or {}
        model_type = data.get("model_type", "random_forest")
        k_neighbors = int(data.get("k_neighbors", 5))
        n_estimators = int(data.get("n_estimators", 100))
        test_size = float(data.get("test_size", 0.25))
        visual_weight = float(data.get("visual_weight", 0.5))

        res = trainer.train(
            model_type=model_type,
            k_neighbors=k_neighbors,
            n_estimators=n_estimators,
            test_size=test_size,
            visual_weight=visual_weight
        )

        return jsonify({
            "success": True,
            "results": res
        })
    except Exception as e:
        traceback.print_exc()
        return jsonify({"success": False, "error": str(e)}), 500

@app.route("/api/evaluate", methods=["POST", "GET"])
```

```
def api_evaluate():
    """
    Generates full performance evaluation:
    - Classifier Accuracy, Precision, Recall, F1, Confusion Matrix
    - Recommender Top-K Hit Rate, MRR, Cosine Similarity, Diversity
    """
    try:
        eval_res = evaluator.run_full_evaluation(trainer)
        return jsonify({
            "success": True,
            "evaluation": eval_res
        })
    except Exception as e:
        traceback.print_exc()
        return jsonify({"success": False, "error": str(e)}), 500

@app.route("/api/recommend", methods=["POST"])
def api_recommend():
    """
    Generates Top-N movie recommendations based on uploaded poster or selected movie.
    """
    try:
        data = request.get_json(silent=True) or {}
        image_input = None
        selected_movie_id = data.get("movie_id")
        top_n = int(data.get("top_n", 6))
        visual_weight = float(data.get("visual_weight", 0.6))

        if "image_base64" in data and data["image_base64"]:
            image_input = data["image_base64"]
        elif selected_movie_id:
            movie = get_movie_by_id(selected_movie_id)
            if movie:
                image_input = os.path.join(POSTERS_DIR, movie["poster_file"])

        if not image_input:
            image_input = os.path.join(POSTERS_DIR, "the_dark_knight.png")

        recs = recommender.recommend_for_image(
            input_image=image_input,
            top_n=top_n,
            visual_weight=visual_weight,
            selected_movie_id=selected_movie_id
        )

        return jsonify({
            "success": True,
            "data": recs
        })
    except Exception as e:
        traceback.print_exc()
        return jsonify({"success": False, "error": str(e)}), 500

@app.route("/api/code-file", methods=["GET"])
def api_get_code_file():
    """Returns the source code of requested pipeline file for viewing/editing in Code Studio."""
    filename = request.args.get("file", "app.py")
```

```
    if filename in ALLOWED_CODE_FILES:
        filepath = ALLOWED_CODE_FILES[filename]
        if os.path.exists(filepath):
            with open(filepath, "r", encoding="utf-8") as f:
                content = f.read()
            return jsonify({
                "success": True,
                "file": filename,
                "content": content
            })
    return jsonify({"success": False, "error": f"File '{filename}' not found or access denied."}), 404

@app.route("/api/run-code", methods=["POST"])
def api_run_code():
    """
    Interactive Code Runner:
    Allows user to type Python ML code in the browser and execute it with live stdout/stderr results!
    """
    try:
        data = request.get_json(silent=True) or {}
        code = data.get("code", "")

        if not code.strip():
            return jsonify({
                "success": False,
                "output": "No code provided to execute."
            })

        # Save code to scratch script
        scratch_dir = os.path.join(BASE_DIR, "scratch")
        os.makedirs(scratch_dir, exist_ok=True)
        script_path = os.path.join(scratch_dir, "user_script.py")
        with open(script_path, "w", encoding="utf-8") as f:
            f.write(code)

        start_time = time.time()
        # Execute script via subprocess with 15s timeout
        proc = subprocess.run(
            [sys.executable, script_path],
            capture_output=True,
            text=True,
            timeout=15,
            cwd=BASE_DIR
        )
        elapsed = round(time.time() - start_time, 3)

        output = proc.stdout
        if proc.stderr:
            output += ("\n--- STDERR ---\n" + proc.stderr) if output else proc.stderr

        return jsonify({
            "success": proc.returncode == 0,
            "output": output if output else "[Execution completed with no terminal output]",
            "returncode": proc.returncode,
            "execution_time_seconds": elapsed
        })
    except subprocess.TimeoutExpired:
```

```
        return jsonify({
            "success": False,
            "output": "[ERROR] Execution timed out after 15 seconds."
        }), 408
    except Exception as e:
        return jsonify({
            "success": False,
            "output": f"[ERROR] Execution failed: {str(e)}"
        }), 500

if __name__ == "__main__":
    if hasattr(sys.stdout, "reconfigure"):
        try:
            sys.stdout.reconfigure(encoding="utf-8")
        except Exception:
            pass
    port = int(os.environ.get("PORT", 5000))
    print(f"\n{'='*70}")
    print(f"  [CINEMATCH] CineMatch Multimodal Movie Recommendation System")
    print(f"  [SERVER] Running locally at: http://127.0.0.1:{port}")
    print(f"  [INFO] All pipeline steps integrated in one frame + Interactive Code Studio")
    print(f"{'='*70}\n")
    app.run(host="0.0.0.0", port=port, debug=False)
```